GEOFLUIDS PROJECT

Investigation of CO_2 and calcite solubilities in the brine using the Gibbs energy minimization approach

Lecturer: Dr. Svetlana Kyas and Dr. Allan Leal

May 13, 2021

1 Introduction

This project is dedicated to chemical equilibrium calculations based on a Gibbs energy minimization approach for the system shown in Table 1. These simulations must be performed with Python. The script must be improved and used to study the solubility calculations of carbon dioxide (CO_2) and calcite ($CaCO_3$) in saline water. The improvements include:

1. Implementation of more accurate models for the activities, a_i, of aqueous and gaseous species to replace the current ones based on ideal solution and ideal gas models. The activity models to be implemented in this project were explained in class, and they are also detailed in Section 2. These activity models are essential for more accurate chemical equilibrium calculations because they are either based on more realistic theoretical models governing the chemistry of fluids or models calibrated with experimental data.

2. The implementation of an interpolation scheme for the calculation of standard chemical potentials of the species, μ_i°, at temperatures and pressures other than only 25 °C and 1 bar. This interpola-

Table 1: Considered chemical system for solubility calculation of CO_2 and $CaCO_3$ in NaCl brine.

Species		Phases		Elements		
α_1	$H_2O(l)$			ε_1	H	*hydrogen*
α_2	$H^+(aq)$			ε_2	O	*oxygen*
α_3	$OH^-(aq)$			ε_3	C	*carbon*
α_4	$HCO_3^-(aq)$			ε_4	Na	*sodium*
α_5	$CO_3^{2-}(aq)$	ϕ_1	Aqueous	ε_5	Cl	*chlorine*
α_6	$CO_2(aq)$			ε_6	Ca	*calcium*
α_7	$Na^+(aq)$			ε_7	Z	*electric charge*
α_8	$Cl^-(aq)$					
α_9	$Ca^{2+}(aq)$					
α_{10}	$CO_2(g)$	ϕ_2	Gaseous			
α_{11}	$CaCO_3$(s, calcite)	ϕ_3	Calcite			

tion capability will extend the temperature and pressure range capability of the code. The latter will permit more accurate equilibrium calculations, i.e., solubility of gases and minerals, at higher temperatures and pressures. The obtained results will allow one to identify how increasing temperature and pressure affect the solubility of CO_2 and $CaCO_3$.

Once the required model improvements have been carried out in the code, use it to perform chemical equilibrium calculations, resulting in an equilibrium state. It contains such essential characteristics as the molar amounts, activities, and concentrations of the species at equilibrium that can be appropriately used to plot the missing curves in the figure templates shown in Figure 1. Each sub-figure of Figure 1 must be generated by the code automating the following processes: *(i)* performing the equilibrium calculation, *(ii)* extracting the appropriate data from the calculated equilibrium state, *(iii)* and generating plots using Python.

2 Implementing activity models for aqueous and gaseous species

Provided Python code calculates the equilibrium state of the chemical system in Table 1 by solving the following Gibbs energy minimization problem:

$$\min_{n} G = \sum_{i=1}^{\mathrm{N}} n_i \mu_i \quad \text{subject to} \quad An = b \quad \text{and} \quad n \geq 0. \tag{1}$$

This minimization problem can be converted into a *system of non-linear equations*, which can then be solved using *Newton's method*. Problem (1) consists of calculating the molar amounts of the species $n = (n_1, \ldots, n_{\mathrm{N}})$ that minimizes the Gibbs energy of the system G. This minimization calculation needs to be performed with two constraints. The first constraint imposes that the amounts of the species n need to satisfy the equation

$$An = b. \tag{2}$$

In other words, the amounts of the species at equilibrium must preserve the given amounts of the elements, here denoted by $b = (b_1, \ldots, b_{\mathrm{E}})$. The second constraint requires that the amounts of the species at equilibrium is non-negative:

$$n \geq 0. \tag{3}$$

Inequality (3) implies that the constraint must be satisfied for each ith species, i.e., $n_i \geq 0$, $i = 1, \ldots, \mathrm{N}$.

The *chemical potential* of the species μ_i is defined as

$$\mu_i = \mu_i^\circ + RT \ln a_i, \tag{4}$$

where μ_i° is the *standard chemical potential* of the ith species, $R = 8.31446\,\mathrm{J/mol}$ is the *universal gas constant*, T is *temperature*, and a_i is the *activity* of the ith species.

The standard chemical potentials of the species μ_i° depend only on temperature T and pressure P. Section 3 provides values for these thermodynamic properties at various temperatures and pressures for all chemical species considered in this project (see Table 3).

The activities of the species a_i can be thought of as their effective concentrations, i.e., their concentrations corrected for the non-ideal behavior of solutions. They are calculated differently for each phase. Section 2.1 describe activity models for aqueous species, and Section 2.2 for CO_2(g).

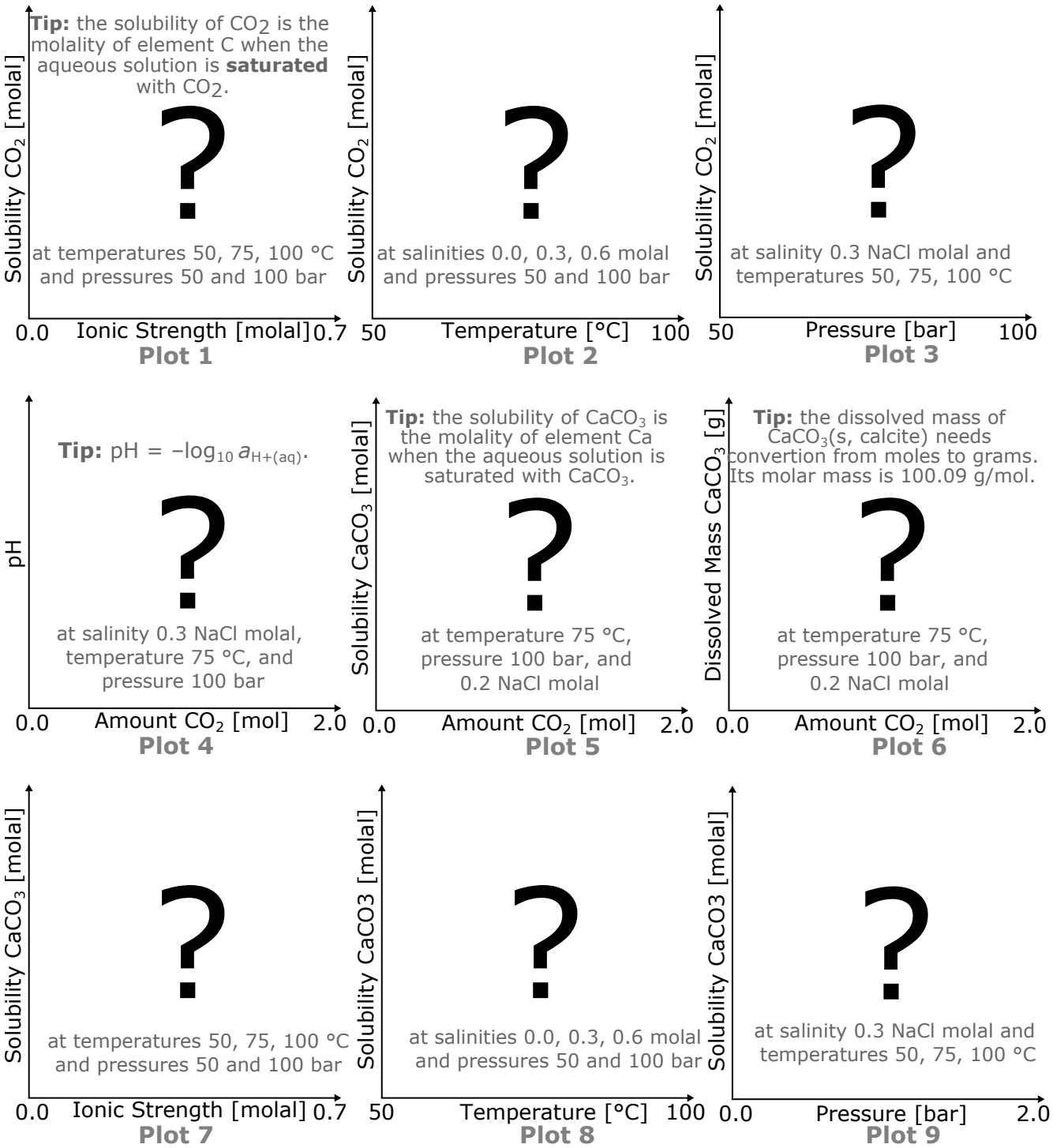

Figure 1: Figures required to be generated in the project.

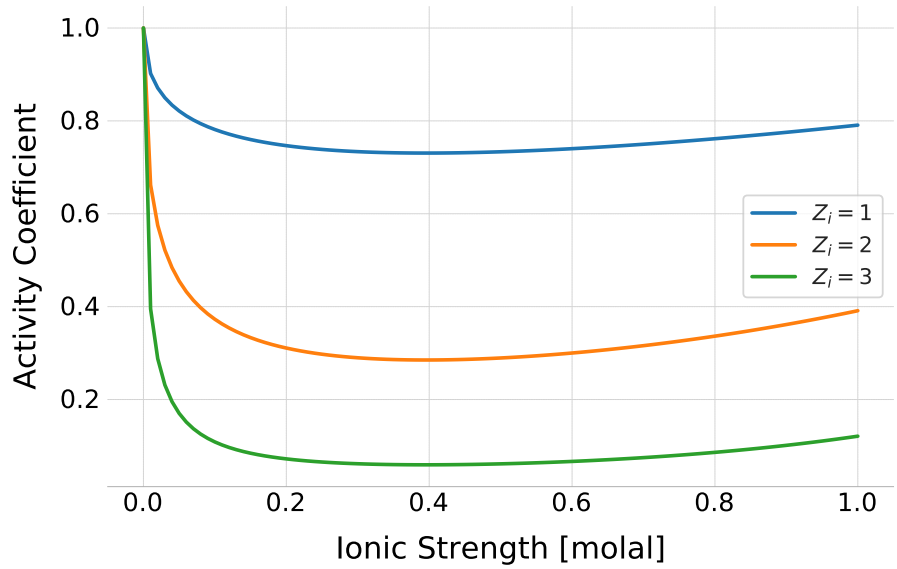

Figure 2: Activity coefficients of ionic species γ_i using the Davies model.

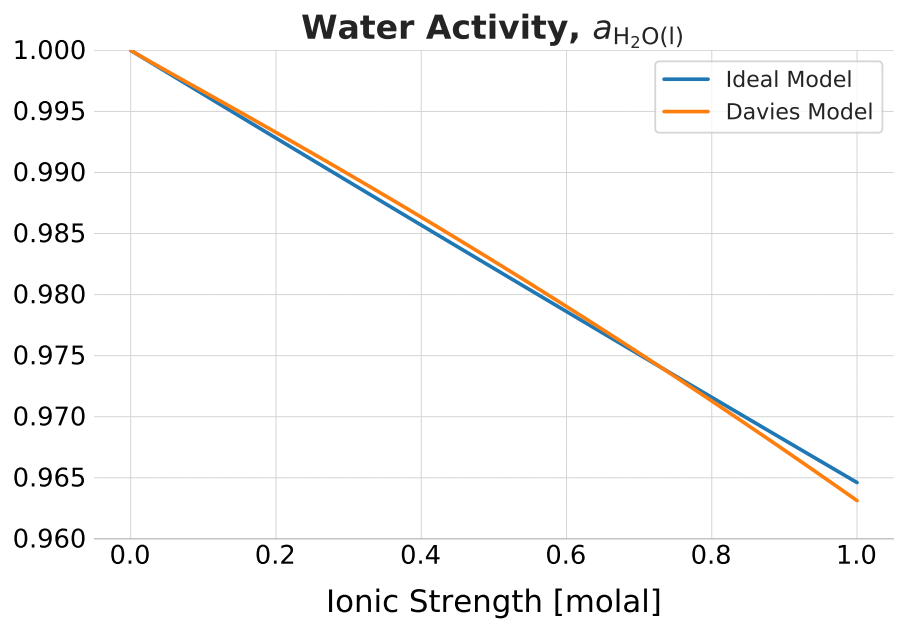

Figure 3: Activity of water species $a_{H_2O(l)}$ using the Davies model and the ideal solution model.

2.1 Activity model for aqueous solution

In an aqueous solution, the species $H_2O(l)$ is referred to as the *solvent species*, and all other species are *solute species*, e.g., $Na^+(aq)$, $HCO_3^-(aq)$, $CO_2(aq)$. The solute species can be *ions*, electrically charged species, such as $H^+(aq)$ and $CO_3^{2-}(aq)$, with charges $+1$ and -2, respectively. Alternatively, they can be *neutral*, such as $CO_2(aq)$. In what follows, we describe *activity models* for aqueous solute species, for both ions and neutral solutes, as well as for aqueous solvent species, $H_2O(l)$. However, before activity models for these species are presented some important definitions of concentration measures for aqueous solutions are presented.

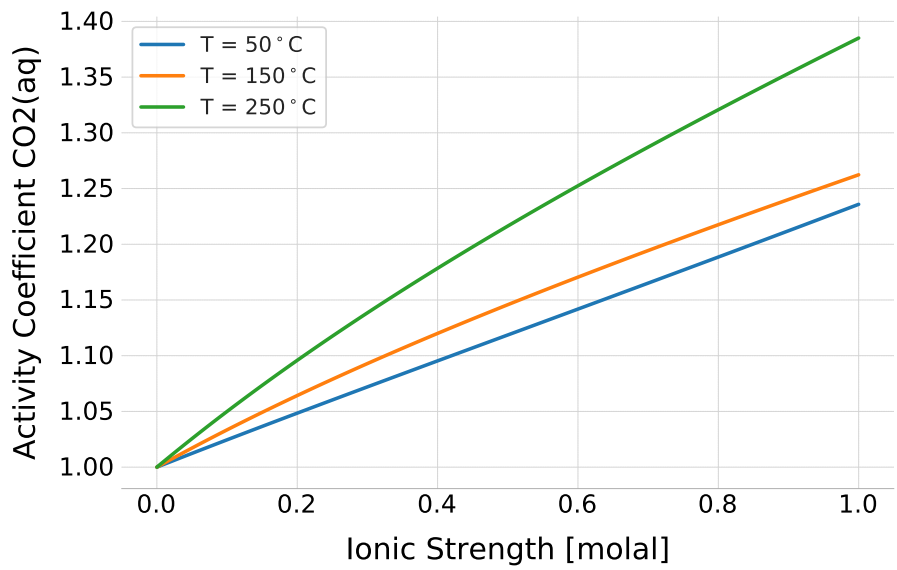

Figure 4: Activity coefficient of dissolved CO_2 species $\gamma_{CO_2(aq)}$ using Drummond (1981) model.

Molality of aqueous solute species

The concentration measure most commonly used for aqueous solutes is *molality*. The molality of a solute is represented by m_i and defined as

$$m_i = \frac{n_i}{M_{H_2O(l)}}, \tag{5}$$

where $M_{H_2O(l)}$ is the *mass of solvent water $H_2O(l)$* in kg. The unit of molality is *molal*, or equivalently mol/kgw, i.e., the number of moles of a solute divided by the mass of solvent water in kg. Using the molar mass of water, $\mathcal{M}_{H_2O} = 18.01528 \cdot 10^{-3}\,\mathrm{kg/mol}$, we can use the following equation instead to calculate the molality of solutes:

$$m_i = \frac{n_i}{\mathcal{M}_{H_2O}\, n_{H_2O(l)}} = 55.5084 \frac{n_i}{n_{H_2O(l)}}. \tag{6}$$

Ionic strength of the aqueous solution

An aqueous solution can exist at infinitely many different compositions (i.e., different concentrations for the solutes). However, when the aqueous solution is sufficiently diluted (with low concentrations of solutes), two such aqueous solutions with different composition tend to behave similarly thermodynamically when their *ionic strength* is the same. The ionic strength of an aqueous solution is defined as

$$I = \frac{1}{2} \sum_{i=1}^{N^{aq}} m_i Z_i^2, \tag{7}$$

where Z_i is the *electrical charge* of the ith aqueous species, and N^{aq} is the number of aqueous species.

Activity of ionic aqueous solute species

The *activity of aqueous ionic species* is defined as

$$a_i = \gamma_i m_i, \tag{8}$$

where γ_i is the *activity coefficient* of the ionic species. The activity coefficient can be seen as a factor that corrects the species concentration for the non-ideal thermodynamic behavior of aqueous solutions.

In this project, the *Davies activity model* needs to be used to calculate γ_i for aqueous ions as follows:

$$\log_{10} \gamma_i = -A_\gamma Z_i^2 \left(\frac{\sqrt{I}}{1+\sqrt{I}} - 0.3I \right), \tag{9}$$

where A_γ is a Debye–Hückel parameter defined as:

$$A_\gamma = 1.824928 \cdot 10^6 \rho_{\mathrm{H_2O}}^{1/2} (\epsilon_{\mathrm{H_2O}} T)^{-3/2}, \tag{10}$$

where $\rho_{\mathrm{H_2O}}$ and $\epsilon_{\mathrm{H_2O}}$ are the *density* and *dielectric* constant of pure water. In this project, use $A_\gamma = 0.5095$, which corresponds to its value at 25°C and 1 bar. The behavior of γ_i with respect of changing ionic strength is illustrated in Figure 2.

Note: To convert from $\log_{10}$ to ln use

$$\ln \gamma_i = \ln 10 \, \log_{10} \gamma_i.$$

Activity of aqueous solvent species H_2O(l)

The activity of the aqueous species H_2O(l) can be calculated using

$$\ln a_{\mathrm{H_2O(l)}} = \frac{\ln 10}{55.5084} A \left[2 \left(\frac{I + 2\sqrt{I}}{1+\sqrt{I}} \right) - 4 \ln(1+\sqrt{I}) - 0.3I^2 \right] - \frac{1 - x_{\mathrm{H_2O(l)}}}{x_{\mathrm{H_2O(l)}}}, \tag{11}$$

where $x_{\mathrm{H_2O(l)}}$ is the mole fraction of H_2O(l). This equation results in activity values for solvent water that satisfies the Gibbs–Duhem equation when the Davies model is used for the activities of ionic species (not addressed in this course). However, we saw in class that for the range of ionic strengths for which the Davies model is valid (i.e., 0.1–0.7 molal) the above equation and the simplification for an ideal solution model

$$\ln a_{\mathrm{H_2O(l)}} = -\frac{1 - x_{\mathrm{H_2O(l)}}}{x_{\mathrm{H_2O(l)}}}. \tag{12}$$

results in very similar values (see Figure 3).

Activity of neutral aqueous solute species CO_2(aq)

The activity of the aqueous species CO_2(aq) can be calculated using

$$a_{\mathrm{CO_2(aq)}} = \gamma_{\mathrm{CO_2(aq)}} m_{\mathrm{CO_2(aq)}}, \tag{13}$$

with the activity coefficient of CO_2(aq) calculated using Drummond (1981) model:

$$\ln \gamma_{\mathrm{CO_2}} = \left(c_1 + c_2 T + \frac{c_3}{T} \right) I - (c_4 + c_5 T) \frac{I}{I+1}, \tag{14}$$

where $c_1 = -1.0312$, $c_2 = 1.2806 \cdot 10^{-3}$, $c_3 = 255.9$, $c_4 = 0.4445$ and $c_5 = -1.606 \cdot 10^{-3}$, T is the temperature in units of K, and I is the ionic strength of the aqueous solution in units of molal. This equation is valid within the temperature and salinity ranges 20–400°C and 0–6.5 molal, respectively. The dependence of the CO_2(aq) activity coefficient $\gamma_{\mathrm{CO_2(aq)}}$ on ionic strength I see in Figure 4.

Table 2: Parameters for the cubic equation of state in Eq. (21) for different models.

Equation of State	$\alpha(T_r, \omega)$	ϵ	σ	Ω	Ψ
Ideal Gas	0	0	0	0	0
vdW[1] (1873)	1	0	0	1/8	27/64
RK[2] (1949)	$T_r^{-1/2}$	0	1	0.08664	0.42748
SRK[3] (1972)	$\left[1 + (0.480 + 1.574\omega - 0.176\omega^2)(1 - T_r^{1/2})\right]^2$	0	1	0.08664	0.42748
PR[4] (1976)	$\left[1 + (0.37464 + 1.54226\omega - 0.26992\omega^2)(1 - T_r^{1/2})\right]^2$	$1 - \sqrt{2}$	$1 + \sqrt{2}$	0.07780	0.45724

[1]van der Waals (1873); [2]Redlich and Kwong (1949); [3]Soave (1972), commonly known as Soave–Redlich–Kwong; [4]Peng and Robinson (1976).

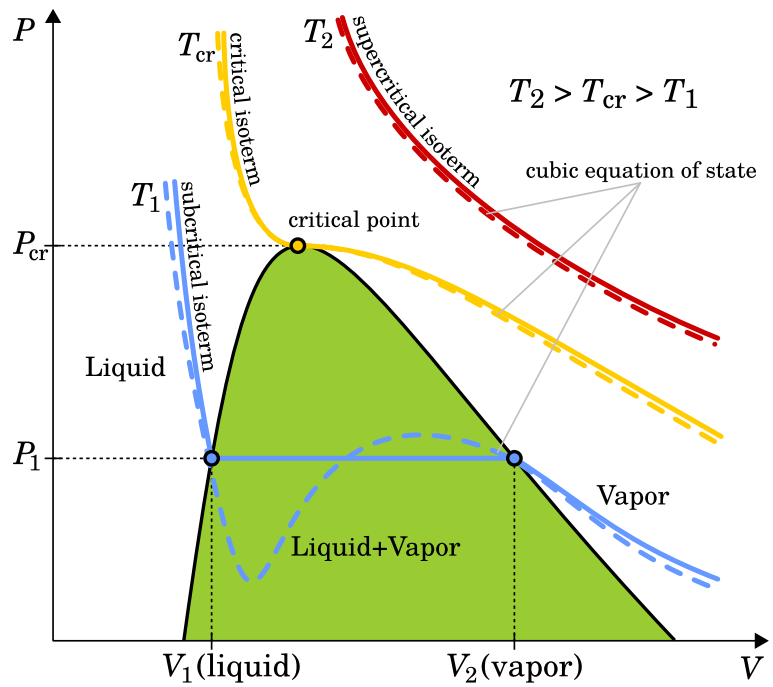

Figure 5: Pressure-volume phase diagram for a substance. Dashed lines correspond to a cubic equation of state modeling the PVT behavior of the substance.

2.2 Activity model for gaseous solution

The activity of a gaseous species is defined as

$$a_i = \frac{\varphi_i x_i P}{P^\circ}, \tag{15}$$

where x_i is the *mole fraction* of the gaseous species in the gaseous solution, φ_i is the *fugacity coefficient* of the species, and P° is a reference pressure, here assumed as $P^\circ = 1\,\text{bar}$. Note that, in this project, we only assume one gas, CO_2(g), so that its mole fraction is one, i.e., $x_i = 1$. When the gaseous solution is an ideal gas mixture, the fugacity coefficient is $\varphi_i = 1$ for all gaseous species.

To correct for the non-ideal thermodynamic behavior of gases, more accurate equations of state need to be used. These equations govern the mutual dependency on temperature T, pressure P, and molar

Algorithm 1 Algorithm for the compressibility factor calculation using fixed-point iteration.

```
function CompressibilyFactor(T, P; T_c, P_c, Ψ, Ω, ω, ϵ, σ)
    T_r ← T / T_c
    P_r ← P / P_c
    α ← α(T_r, ω)                          ▷ See Table 2 for function α(T_r, ω) for different models.
    β ← Ω P_r / T_r                        ▷ Using Eq. (22).
    q ← (Ψ/Ω) α / T_r                      ▷ Using Eq. (23).
    Z ← 1                                  ▷ Initial guess for Z corresponding to the compressibility factor of an ideal gas.
    f(Z) ← 1 + β − qβ (Z − β)/((Z + ϵβ)(Z + σβ))   ▷ Define f(Z) to be used in the fixed-point algorithm.
    FixedPoint(Z, f(Z))                    ▷ Call fixed-point iterative algorithm.
```

volume V with the equation of state for an ideal gas being the simplest one:

$$\frac{PV}{RT} = 1.$$

In this project, the following *general cubic equation of state* needs to be used (see Smith et al., 2005, p. 93, eq. 3.42):

$$P = \frac{RT}{V-b} - \frac{a(T,\omega)}{(V+\epsilon b)(V+\sigma b)}, \tag{16}$$

where ϵ and σ are given constants (see their values for different equation of state models in Table 2). Here, a and b are

$$a(T) := \Psi \frac{\alpha(T_\mathrm{r},\omega) R^2 T_\mathrm{c}^2}{P_\mathrm{c}}, \tag{17}$$

and

$$b := \Omega \frac{RT_\mathrm{c}}{P_\mathrm{c}} \tag{18}$$

with T_c and P_c denoting the *critical temperature* and *critical pressure* of the gas, Ψ and Ω are given constants, and $\alpha(T_\mathrm{r},\omega)$ is a given function, each of these varying with the equation of state model (see Table 2); T_r and P_r denoting the *reduced temperature* and *reduced pressure*, of the substance

$$T_\mathrm{r} = \frac{T}{T_\mathrm{c}} \qquad \text{and} \qquad P_\mathrm{r} = \frac{P}{P_\mathrm{c}}, \tag{19}$$

respectively. Finally, ω is the *acentric factor* of the substance, needed in the evaluation of function $\alpha(T_\mathrm{r},\omega)$ for some models (see Table 2), with $\omega = 0.224$ for CO_2.

Every substance has a different critical temperature T_c and pressure P_c. When the temperature and pressure of the substance is greater than their respective critical values, the substance is said to be in a *supercritical state*. This is a physical state in which the substance behaves similarly to both liquid and gas (e.g., the substance is dense, like a liquid, and has low viscosity, like a gas). The critical temperature and pressure of CO_2 are 304.2 K and 73.83 bar, respectively.

Equation (16) can be more conveniently written in terms of the *compressibility factor* Z of the gaseous

Algorithm 2 Algorithm for the compressibility factor calculation using Newton iteration.

function COMPRESSIBILYFACTOR($T, P; T_c, P_c, \Psi, \Omega, \omega, \epsilon, \sigma$)
  $T_r \leftarrow T/T_c$
  $P_r \leftarrow P/P_c$
  $\alpha \leftarrow \alpha(T_r, \omega)$ ▷ See Table 2 for function $\alpha(T_r, \omega)$ for different models.
  $\beta \leftarrow \Omega \frac{P_r}{T_r}$ ▷ Using Eq. (22).
  $q \leftarrow \frac{\Psi}{\Omega} \frac{\alpha}{T_r}$ ▷ Using Eq. (23).
  $Z \leftarrow 1$ ▷ Initial guess for Z corresponding to the compressibility factor of an ideal gas.
  $r(Z) \leftarrow 1 + \beta - q\beta \frac{Z-\beta}{(Z+\epsilon\beta)(Z+\sigma\beta)} - Z$ ▷ Define residual function for the Newton method.
  $r'(Z) \leftarrow -\frac{q\beta}{(Z+\epsilon\beta)(Z+\sigma\beta)} \left\{1 - (Z-\beta)\frac{2Z+(\epsilon+\sigma)\beta}{(Z+\epsilon\beta)(Z+\sigma\beta)}\right\} - 1$ ▷ Define residual function's derivative.
  NEWTON($Z, r(Z), r'(Z)$) ▷ Call Newton iterative algorithm.

Algorithm 3 Fixed-point method.

function FIXEDPOINT($Z, f(Z)$)
  $i_{\max} = 100$ ▷ Maximum number of iterations.
  $\epsilon_{\text{tol}} = 1e-4$ ▷ Error tolerance.
  for $i \leftarrow 1$ to $i_{\max}$ **do**
    $Z_{\text{new}} \leftarrow f(Z)$ ▷ Using Eq. (24).
    if $|Z_{\text{new}} - Z|/Z < \epsilon_{\text{tol}}$ **then** ▷ We stop when Z_{new} is relatively close enough to Z.
      return Z_{new}
    $Z \leftarrow Z_{\text{new}}$
  throw *Error: Calculation failed to converge.*

solution, i.e.,

$$Z := \frac{PV}{RT}. \tag{20}$$

After some algebraic manipulations, the following equation can be derived in terms of Z only:

$$Z = \frac{Z}{Z-\beta} - q\beta \frac{Z}{(Z+\epsilon\beta)(Z+\sigma\beta)}, \tag{21}$$

where β and q are defined as

$$\beta := \Omega \frac{P_r}{T_r} \tag{22}$$

and

$$q := \frac{\Psi}{\Omega} \frac{\alpha(T_r, \omega)}{T_r}. \tag{23}$$

We will use Eq. (21) to calculate Z for a given temperature T and pressure P. Since Eq. (21) is a cubic

Algorithm 4 Newton's method.

```
function NEWTON(Z, r(Z), r'(Z))
    i_max = 100                                  ▷ Maximum number of iterations.
    ε_tol = 1e − 4                               ▷ Error tolerance.
    for i ← 1 to i_max do
        Z_new ← Z − r_Z / r'_Z
        if r_Z < ε_tol then                      ▷ We stop when residual r(Z) is sufficiently close to zero.
            return Z_new
        Z ← Z_new
    throw Error: Calculation failed to converge.
```

polynomial

$$Z^3 + AZ^2 + BZ + C = 0,$$

it follows that there exists at most three values (i.e., three roots) for Z that satisfies (2.2). When these three roots are real numbers, the smallest value corresponds to a liquid state, and the largest value corresponds to a gas state. There also could be only one real root and a conjugate pair of complex roots! The middle value is not physically relevant (see Figure 5). Since we are interested in calculating the fugacity coefficient of CO_2(g) in this project, we should then require only the largest root of Eq. (21). For this, it is numerically convenient to write Eq. (21) in the following form (see Smith et al., 2005, p. 97, eq. 3.52):

$$Z = f(Z) = 1 + \beta - q\beta \frac{Z - \beta}{(Z + \epsilon\beta)(Z + \sigma\beta)}. \tag{24}$$

A simple *fixed-point iterative algorithm* to find Z that solves Eq. (24) is given in Algorithm 1. This method, however, is not very efficient and robust. Thus, *Newton's method* is preferred.

In Newton's method, we transform the non-linear equation in Eq. (24) into the form

$$r(Z) = f(Z) - Z = 0, \tag{25}$$

where r is the *residual function* defined as

$$r(Z) = 1 + \beta - q\beta \frac{Z - \beta}{(Z + \epsilon\beta)(Z + \sigma\beta)} - Z. \tag{26}$$

Newton's method requires the first-order derivative $r'(Z)$

$$r'(Z) = -\frac{q\beta}{(Z + \epsilon\beta)(Z + \sigma\beta)} \left\{ 1 - (Z - \beta) \frac{2Z + (\epsilon + \sigma)\beta}{(Z + \epsilon\beta)(Z + \sigma\beta)} \right\} - 1. \tag{27}$$

The iterative procedure in Newton's method is as follows: start with an initial guess for Z, say $Z^0 = 1$.

Then, perform the following iterative steps:

$$\begin{aligned} Z^1 &= Z^0 - r(Z^0)/r'(Z^0) && 1^{\text{st}} \text{ iteration} \\ &\vdots && \vdots \\ Z^{i+1} &= Z^i - r(Z^i)/r'(Z^i) && i^{\text{th}} \text{ iteration} \end{aligned} \tag{28}$$

until iteration i satisfies

$$r(Z^{i+1}) < \epsilon_{\text{tol}}, \tag{29}$$

where ϵ_{tol} is a tolerance for the residual function r. See Algorithm 4 for a more detailed description of the required steps to calculate Z using Newton's method.

Once Z is calculated, we can calculate the fugacity coefficient of the gas using

$$\boxed{\ln \varphi_i := Z - 1 - \ln(Z - \beta) - q\theta}, \tag{30}$$

where θ is defined as

$$\theta := \begin{cases} \dfrac{1}{\sigma - \epsilon} \ln\left(\dfrac{Z + \sigma\beta}{Z + \epsilon\beta}\right) & \epsilon \neq \sigma, \\ \dfrac{\beta}{Z + \epsilon\beta} & \epsilon = \sigma. \end{cases} \tag{31}$$

3 Implementing interpolation function for standard chemical potentials

The provided Python code is limited to calculate the solubility of CO_2 and $CaCO_3$ at 25 °C and 1 bar only, since the code only uses *standard chemical potentials of the species* μ_i° at these temperature and pressure values.

Table 3 contains the standard chemical potentials of all species in Table 1 at temperatures 25, 50, 75, and 100 °C and pressures 1, 50, 100 bar. To perform needed calculations at a given T and P, , μ_i° needs to be interpolated from the values in the table. In what follows, we describe how this interpolation calculation can be performed in general.

Let $f(x, y)$ be a function defined over the region $[x_L, x_R] \times [y_B, y_T]$ and let f_{ij} denote its tabulated value at the discrete coordinate point (x_i, y_j) (see Figure 6). Here, x_i denotes the x-coordinate of the ith point in the discretization of the range $[x_L, x_R]$, and y_j corresponds to the y-coordinate of the jth point in the the discretization of the range $[y_B, y_T]$. If (x, y) is the point where the value of f needs to be interpolated, i.e., where $f(x, y)$ is needed, with $x \in [x_L, x_R]$ and $y \in [y_B, y_T]$, then one can use the following first order Lagrange interpolation formula

$$f(x, y) = f_{i,j} + \frac{x - x_i}{x_{i+1} - x_i}(f_{i+1,j} - f_{i,j}) + \frac{y - y_j}{y_{j+1} - y_j}(f_{i,j+1} - f_{i,j}). \tag{32}$$

The latter can be used to calculate $\mu_i^\circ(T, P)$ using tabulated values of μ_i° over a set of given temperature and pressure points. The coordinates x and y above correspond to the temperature and pressure, whereas the function f stands for the standard chemical potential of the species μ_i°.

Table 3: The standard chemical potentials of the chemical species μ_i° considered in this project, in J/-mol. These values were calculated by Reaktoro (Leal, 2015) using the SUPCRT92 (Johnson et al., 1992) database `slop98.dat`.

	T [Â°C]	P [bar]		
		1	50	100
H_2O(l)	25	-237181.72	-237093.28	-237003.23
	50	-239006.71	-238917.46	-238826.59
	75	-240977.57	-240887.12	-240795.03
	100	—	-242992.18	-242898.53
H^+(aq)	25	0	0	0
	50	0	0	0
	75	0	0	0
	100	—	0	0
OH^-(aq)	25	-157297.48	-157320.08	-157342.16
	50	-156905.20	-156924.41	-156943.13
	75	-156307.81	-156328.46	-156348.60
	100	—	-155558.52	-155583.77
HCO_3^-(aq)	25	-586939.89	-586820.91	-586698.78
	50	-589371.68	-589247.85	-589120.88
	75	-591758.94	-591634.79	-591507.48
	100	—	-593983.38	-593858.95
CO_3^{2-}(aq)	25	-527983.14	-528011.90	-528039.31
	50	-526461.60	-526494.20	-526525.56
	75	-524473.15	-524514.70	-524555.01
	100	—	-522121.51	-522175.91
CO_2(aq)	25	-385974.00	-385813.47	-385650.13
	50	-389147.04	-388979.61	-388809.39
	75	-392728.17	-392556.66	-392382.38
	100	—	-396484.88	-396307.90
Na^+(aq)	25	-261880.74	-261886.18	-261890.73
	50	-263384.24	-263384.84	-263384.60
	75	-264979.92	-264978.35	-264975.97
	100	—	-266664.71	-266661.71
Cl^-(aq)	25	-131289.74	-131204.66	-131117.63
	50	-132590.59	-132504.28	-132416.09
	75	-133682.46	-133598.32	-133512.30
	100	—	-134501.87	-134420.82
Ca^{2+}(aq)	25	-552790.08	-552879.51	-552968.88r
	50	-551348.98	-551437.62	-551526.28
	75	-549855.33	-549945.95	-550036.62
	100	—	-548400.71	-548495.68
CO_2(g)	25	-394358.74	-394358.74	-394358.74
	50	-399740.71	-399740.71	-399740.71
	75	-405197.73	-405197.73	-405197.73
	100	-410726.87	-410726.87	-410726.87
$CaCO_3$(s,calcite)	25	-1129177.92	-1128996.94	-1128812.26
	50	-1131580.05	-1131399.06	-1131214.39
	75	-1134149.89	-1133968.91	-1133784.23
	100	-1136882.60	-1136701.62	-1136516.94

Note: At 100°C and 1 bar, water is in vapor state, and thus a dash "—" is used for the values of μ_i° for aqueous species at that temperature and pressure condition. What could be an appropriate value to use in the Python code for those entries?

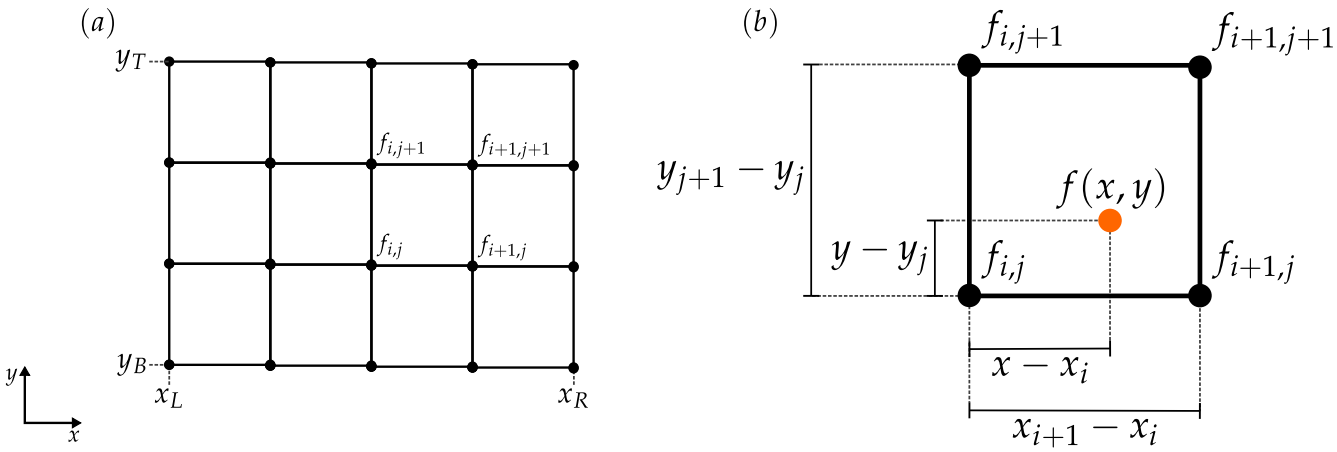

Figure 6: Illustration of the tabulated values of function f and its interpolation using these values. *(a)* The grid where the values of function f are tabulated over the black bullet points. *(b)* The use of the first order Lagrange interpolation scheme to interpolate f at the red point at (x, y).

Temperature [K]	Temperature [C]	Pressure [Pa]	Pressure [bar]
298.15	25.0	100000.0	1.0

Species	Amount [mol]	Concentration	Activity	Activity Coefficient
H2O(l)	55.5015468188	0.991898428122	0.991865521948	0.999966825057
H+(aq)	1.18005842842e-06	1.18019563416e-06	1.18019563416e-06	1.0
OH-(aq)	8.49827403557e-09	8.4992621324e-09	8.4992621324e-09	1.0
HCO3-(aq)	0.0129051738875	0.0129066743759	0.0129066743759	1.0
CO3--(aq)	5.12897569547e-07	5.12957204299e-07	5.12957204299e-07	1.0
CO2(aq)	0.0339630054716	0.0339669543602	0.0339669543602	1.0
Na+(aq)	0.2	0.200023254059	0.200023254059	1.0
Cl-(aq)	0.2	0.200023254059	0.200023254059	1.0
Ca++(aq)	0.00645251406127	0.00645326429698	0.00645326429698	1.0
CO2(g)	3.9595838218	1.0	1.0	1.0
CaCO3(s,calcite)	0.00354748593873	1.0	1.0	1.0

Figure 7: The file `results.txt` that is output by executing the Python code `project.py`.

4 Explanation about the Python code

The provided Python code, `project.py`, can be executed in your machine or using some website that offers a development environment for Python, such as `repl.it` at

`https://repl.it/languages/python.`

If you prefer to run it on your local machine, you will need to install Python 3, if it is not already natively provided by your operating system (e.g., by Linux and MacOS). For Windows users, you can find the Python installer at

`https://www.python.org/downloads/release/python-2713/`.

Once Python is available in your machine, you can execute the file `project.py` in a terminal using

```
$ python project.py
```

with the assumption that the current directory in the terminal is where the file `project.py` is located. If not, use the command `cd` to change to the correct directory. Alternatively, you can install the SublimeText editor, open `project.py`, and press Ctrl + b.

Whichever way you will execute the source-code, a `results.txt` file should be output in the same direc-

tory. Its content should be similar to what is shown in Figure 7. The output file shows the temperature and pressure inputs and the resulting equilibrium state, listing the molar amounts, concentrations, activities, and activity coefficients of the species. Note that in the output file that activity coefficients γ_i of the aqueous species and the fugacity coefficient φ_i of CO_2(g) is one (last column of the table in Figure 7), so that the calculated equilibrium state should not be accurate since the thermodynamic behavior of the aqueous solution and CO_2(g) is modeled using ideal models.

At the bottom of `project.py` file, you should see a function with the following signature and return statement:

```
def equilibrate(T, P, b):
    ...
    return equilibriumstate
```

Listing 1: Function of chemical equilibrium algorithm

In this function, `T` is the temperature in K, `P` is the pressure in Pa, and `b` is the array with the molar amounts of the elements. The returned object, `equilibriumstate`, contains the temperature and pressure used in the calculation, the amounts, concentrations, activities, and activity coefficients of the species. The following is the usage to perform an equilibrium calculation and the subsequent calls:

```
# Perform an equilibrium calculation with given T, P, b
# and store the result in object ‘state‘.
# Here ‘state‘ is just a name to the object containing
# the calculated equilibrium state data.
# One could have chosen a different name, like ‘mystate‘.
state = equilibrate(T, P, b)

# To extract data from object ‘state‘, use the following statements:
state.T  # access the temperature stored in the returned ‘state‘ object
state.P  # access the pressure stored in the returned ‘state‘ object
state.n  # access the molar amounts (array) of the species in ‘state‘
state.c  # access the concentrations (array) of the species in ‘state‘
state.a  # access the activities (array) of the species in ‘state‘
state.g  # access the activity coefficients (array) in ‘state‘
```

Listing 2: Perform an equilibrium calculation

Because we want to perform equilibrium calculations using inputs of the form: 1 kg of H_2O, 1 mol of CO_2, 0.5 moles of NaCl, 5 moles of $CaCO_3$, `project.py` provides the utility function:

```
def component_amounts(kgH2O, molCO2, molNaCl, molCaCO3):
...
return b
```

Listing 3: component_amounts() function

which can be used in the example to calculate the pH of the aqueous solution:

```
T = 333.15  # temperature in K (=60 C)
P = 100e5  # pressure in Pa (=100 bar)
# Calculate the amounts of elements based on a mixture of
# 1 kg of H2O, 1 mol of CO2, 0.5 moles of NaCl, and 5 moles of CaCO3.
b = component_amounts(kgH2O=1, molCO2=1, molNaCl=0.5, molCaCO3=5)
state = equilibrate(T, P, b)  # perform the equilibrium calculation
iH = species.index('H+(aq)')  # the index of species H+(aq)
pH = -log10(state.a[iH])  # calculate the pH of the aqueous solution
print 'The pH in the calculated equilibrium state is', pH
```

Listing 4: Calculate pH

One of the figures to be plotted as part of this project is a pH versus amount of CO_2 (see Plot4 in Figure 1). Here, a series of equilibrium calculations need to be performed at 75°C and 100 bar. In this series of calculations, the idea is to mix an increasing amount of CO_2, from 0 to 2 moles, with 1 kg of H_2O and 0.3 mol of NaCl (which should produce an aqueous solution with 0.3 NaCl molal as required). You should decide how many increasing steps for the amount of CO_2. At each calculation, the pH of the solution should then be calculated by accessing the calculated equilibrium state data; collecting the activity of species H^+(aq), $a_{H^+(aq)}$; and then calculating $\text{pH} = -\log_{10}(a_{H^+(aq)})$. For this plot, the x-axis is the amount of CO_2, varying from 0 to 2 moles, and the y-axis is the calculated pH.

For all other figures, a similar procedure should be performed: perform a series of equilibrium calculations by adequately varying the inputs (e.g., temperature, pressure, amounts of CO_2, NaCl, $CaCO_3$, etc.) and then extract from the calculated equilibrium state the proper quantities that will serve for its immediate use in a plot or for calculating another quantity of interest (e.g., pH).

References

Drummond, S. (1981). *Boiling and mixing of hydrothermal fluids: chemical effects on mineral precipitation*. Ph.d., Pennsylvania State University.

Johnson, J. W., Oelkers, E. H., and Helgeson, H. C. (1992). SUPCRT92: A software package for calculating the standard molal thermodynamic properties of minerals, gases, aqueous species, and reactions from 1 to 5000 bar and 0 to 1000 C. *Computers & Geosciences*, 18(7):899–947.

Leal, A. M. (2015). Reaktoro: A unified framework for modeling chemically reactive systems.

Peng, D.-Y. and Robinson, D. B. (1976). A New Two-Constant Equation of State. *Industrial & Engineering Chemistry Fundamentals*, 15(1):59–64.

Redlich, O. and Kwong, J. N. S. (1949). On the Thermodynamics of Solutions. V. An Equation of State. Fugacities of Gaseous Solutions. *Chemical Reviews*, 44(1):233–244.

Smith, J. M., Ness, H. C. V., and Abbott, M. M. (2005). *Introduction to chemical engineering thermodynamics*. McGraw-Hill.

Soave, G. (1972). Equilibrium constants from a modified Redlich-Kwong equation of state. *Chemical Engineering Science*, 27(6):1197–1203.

van der Waals, J. D. (1873). *On the Continuity of the Gaseous and Liquid States*. PhD thesis, Universiteit Leiden.